



Python Programming – Week 10: Files

Exercise 1 — Copy with Line Numbers (Text → Text)

Task: Read from `input.txt`, line by line. Write the line to `output.txt` where each line is prefixed with its line number starting from 1.

Requirement: Keep the original line content (except adding the number).

Exercise 2 — Count Lines, Words, Characters (Text → Text Report)

Task: Read from `input.txt`. Compute:

1. number of lines
2. total number of words (split by spaces)
3. total number of characters (including spaces)

Write the results as a 3-line report to `report.txt`.

Exercise 3 — Keep Only Non-Empty Lines

Task: Read from `input.txt` and write to `clean.txt` only the lines that are not empty after stripping spaces.

Exercise 4 — Uppercase All Lines

Task: Read from `input.txt`. Write the lines to `upper.txt` where all letters are converted to uppercase.

Exercise 5 — Extract Lines Containing a Keyword

Task: Ask the user for a keyword in the main program.

Read from `input.txt` and write to `matches.txt` only the lines that contain the keyword (case-sensitive).

Exercise 6 — Sum Numbers from a Text File

Task: Read from `numbers.txt`, where each line contains one integer.
Compute the total sum and write it to `sum.txt` as:

`Total: <sum>`

Exercise 7 — Filter Numbers by Threshold

Task: Ask the user for an integer threshold `T`.
Read from `numbers.txt` (one integer per line).
Write to `filtered_numbers.txt` only numbers greater than or equal to `T`.
Also write the count of kept numbers at the top of the output file.

Exercise 8 — CSV: Copy Selected Columns

Task: Given `students.csv` with columns: `name, age, major, gpa`
Write `students_simple.csv` containing only `name` and `gpa` columns.
Requirement: Keep the header row.

Exercise 9 — CSV: Compute a New Column

Task: Given `sales.csv` with columns: `item, price, quantity`
Create `sales_total.csv` containing: `item, price, quantity, total`
where `total = price * quantity`.
Requirement: Keep header row and preserve item names.

Exercise 10 — CSV: Count Rows Matching a Condition

Task: Given `students.csv` with columns: `name, age, major, gpa`
Count how many students have `gpa >= 3.0`.
Write the result to `gpa_report.txt` as one line:
`Number of students with GPA >= 3.0: <count>`

For each exercise below:

- ✓ Write **one or more functions**
- ✓ Call them from the **main program**
- ✓ Read from file(s) and write output file(s)

Exercise 11 — Function: Count Keyword Occurrences (Text)

Write a function `count_keyword_in_file(input_path, keyword)` that returns how many lines contain the keyword.

In main:

1. ask the user for keyword
2. call the function on `input.txt`
3. write the count to `keyword_count.txt`

Exercise 12 — Function: Longest Line Finder

Write a function `longest_line(input_path)` that returns:

- the longest line (without trailing newline)
- its line number (starting from 1)

In main, call it on `input.txt` and write a report to `longest_line.txt`.

Exercise 13 — Functions: Normalize Whitespace

Write:

- `normalize_line(line)` → replaces multiple spaces with a single space and strips leading/trailing spaces
- `normalize_file(input_path, output_path)` → reads input file and writes normalized lines

In main, call `normalize_file("input.txt", "normalized.txt")`.

Exercise 14 — Function: Merge Two Text Files Alternating Lines

Write a function `merge_alternating(file1, file2, out_file)` that writes:

- line 1 of file1, line 1 of file2, line 2 of file1, line 2 of file2, ...

In main, call it to merge `a.txt` and `b.txt` into `merged.txt`.

Requirement: If one file ends early, append the remaining lines of the other file.

Exercise 15 — Functions: Validate Numbers File

`numbers.txt` is supposed to contain one integer per line, but some lines may be invalid.

Write:

- `is_valid_int(s)` → returns True if `s` can be parsed as an integer
- `split_valid_invalid(input_path, valid_out, invalid_out)` → writes valid integers to `valid.txt` and invalid lines to `invalid.txt`

In main, call `split_valid_invalid(...)` on `numbers.txt`.

Exercise 16 — Function: CSV Filter by Column Condition

Write a function `filter_csv_by_min_value(input_csv, output_csv, column_name, min_value)` that keeps only rows where the numeric column `column_name` $\geq$ `min_value`.

In main:

1. ask user for `min_value`
 2. call the function on `students.csv` for column `gpa`
 3. write results to `students_gpa_filtered.csv`
-

Exercise 17 — Functions: CSV Group Count

Given `logs.csv` with columns: `user`, `action` (one action per row).

Write:

- `count_actions(input_csv)` → returns a dictionary mapping `action` -> `count`
- `write_action_report(counts, output_path)` → writes a text report sorted by action name

In main, call the functions and produce `action_report.txt`.

Exercise 18 — Functions: CSV Compute Average per Group

Given `scores.csv` with columns: `student`, `subject`, `score`

Write:

- `average_score_per_subject(input_csv)` → returns a dictionary `subject -> average_score`
- `write_subject_averages(avg_dict, output_path)` → writes results to `subject_averages.txt`

In main, call them and create the report.

Exercise 19 — Functions: Detect Duplicate IDs in CSV

Given `employees.csv` with columns: `id, name, department`

Write:

- `find_duplicate_ids(input_csv)` → returns a set (or list) of IDs that appear more than once
- `write_duplicates(duplicates, output_path)` → writes them to `duplicate_ids.txt`, one per line

In main, call the functions and write the report.

Exercise 20 — Mini Project: Clean + Summarize CSV

Given `transactions.csv` with columns: `date, category, amount`

Some rows may have:

- missing category
- invalid amount (not a number)

Write these functions:

1. `is_valid_amount(s)` → checks whether `s` is a valid number
2. `clean_transactions(input_csv, clean_csv, bad_rows_txt)` → writes valid rows to `clean_transactions.csv`, invalid rows to `bad_rows.txt`
3. `sum_by_category(clean_csv)` → returns dictionary `category -> total_amount`
4. `write_category_summary(summary, out_txt)` → writes a neat summary report

In main:

- call cleaning

- then compute totals per category
- output `category_summary.txt`